Phase 2 | Codebase Deep Dive

Follow the implementation from application startup through routes, models, and browser components.

React/Vite Edition | Repository-grounded study material | Snapshot: 2026-10-24

1. Repository reading order

Path	What to learn
README.md	Local run instructions, environment names, deployment notes and stack summary.
frontend/src/main.jsx and App.jsx	React bootstrap and client-side route registration.
frontend/src/app/context/AuthContext.jsx	Login/register state, localStorage persistence, logout and token refresh calls.
backend/index.js	Express middleware order, route mounts, OAuth callback and static file serving.
backend/routes/authRoutes.js	Registration, login and refresh endpoint behavior.
backend/models/User.js	User schema, bcrypt pre-save hook and JWT signing method.
backend/middleware/authMiddleware.js	Bearer-token verification and student/teacher role guards.
backend/routes/alertRoutes.js	SACHET summary normalization and CAP XML parsing.
backend/routes/nearbyRoutes.js	Geoapify request, cache, coalescing and provider error mapping.
flask/app.py	Chat endpoint, system instruction and Gemini request.
frontend/public/earthquake_drill/game.js	Phaser scenes, controls, collision logic and results.

2. Startup and service boundaries

Express loads dotenv, configures CORS, JSON parsing, sessions and Passport, attempts MongoDB connection, mounts API routes, serves uploads and the Vite build, then listens on PORT or 5000. During development, Vite typically runs on 5173 and proxies /api and /uploads to Express. Flask is launched separately on 5001. The standalone quiz has its own package and dev server.

3. Client route map

Route family	React page/module
/	Landing page
/login and /register	Authentication forms
/dashboard/student and /dashboard/teacher	Main learning dashboards
/dashboard/student/earthquakes	Earthquake learning content and assignments
/dashboard/student/nearby	Emergency services list and map
/dashboard/chatbot	Flask/Gemini chat UI
/alerts/cap/:identifier	Readable CAP alert details

4. API contracts and data ownership

Auth routes return JSON user properties and a signed token; profile routes require a Bearer token. Alert summaries are normalized to a small JSON structure with ID, event, time, location, severity and link. CAP XML is parsed into description, instruction, timing and affected areas. Nearby features are converted into element-like records with coordinates and category tags. Chat uses JSON messages and responses.

5. Persistence and uploads

The Mongoose User model stores name, username, email, password, profile image path, role and optional Google ID, with timestamps. Passwords are hashed before save. Multer stores profile images under uploads and the Express service exposes that directory. Upload size limits and stronger content verification should be added.

6. Route inventory

Endpoint	Purpose
POST /api/auth/register	Create account; optional profile image; issue JWT.
POST /api/auth/login	Verify password and issue JWT.
GET /api/auth/google and callback	Passport Google OAuth flow.
POST /api/auth/refresh	Attempt token renewal.
GET /api/users/profile	Protected profile read.
DELETE /api/users/delete-account	Protected account deletion.
GET /api/alerts and /api/alerts/cap/:id	SACHET summary and CAP detail.
GET /api/nearby	Nearby POI lookup.
POST /api/upload	Profile image upload.

Repository behavior to verify before a demo: external API keys and service URLs are environment-specific. A successful local frontend build does not prove the Express, MongoDB, Geoapify or Flask integrations are configured in the deployed environment.